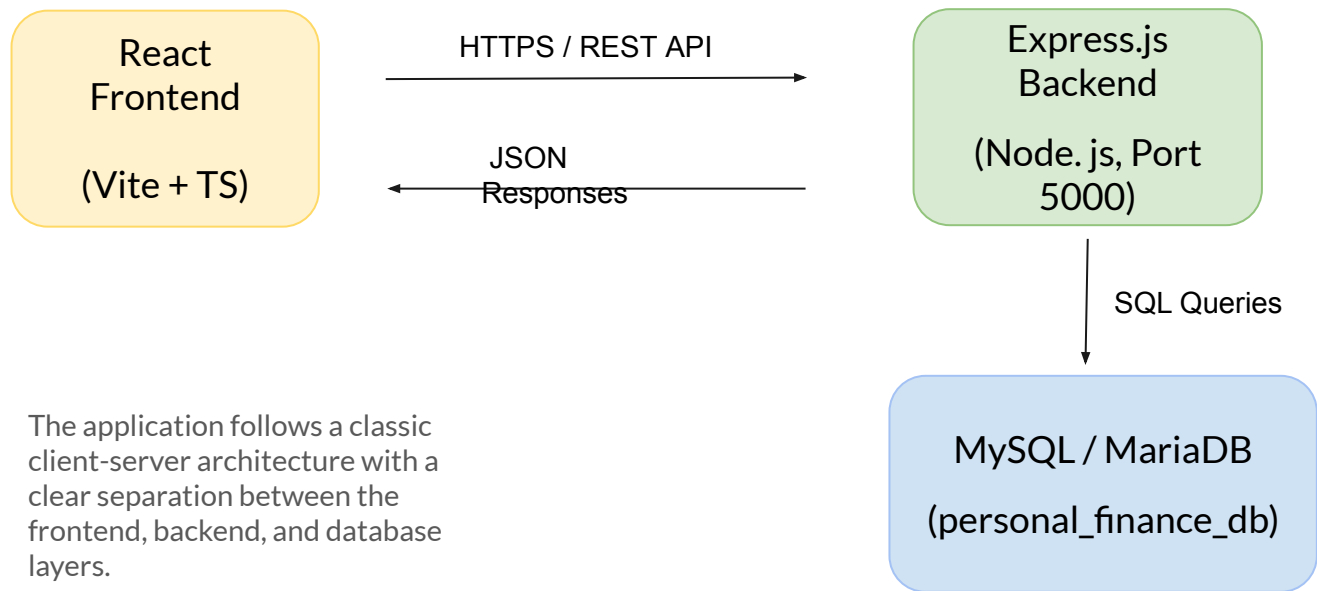




FinTrack - Personal Finance Dashboard

Sisenco Digital | Full-Stack Developer Technical Assignment

System Architecture Overview (Contd.)





Key architectural decisions

- **Stateless API** - Authentication is handled via JWT tokens, with no server-side sessions. Every request is self-contained.
- **Single aggregated dashboard endpoint** - Instead of multiple round trips, a single `/api/dashboard` endpoint returns all data the dashboard needs in one call, reducing latency.
- **Soft deletes** - Transactions and categories are never physically removed. An `is_deleted` flag preserves data integrity for budgets and historical records.
- **Environment-driven configuration** - No hardcoded credentials. Both frontend and backend rely entirely on `.env` files.



Database Design Explanation (Contd.)

The schema consists of four tables with clear ownership boundaries through ***user_id*** foreign keys.

```
users
├── categories (user_id → users.id)
│   ├── transactions (user_id → users.id, category_id → categories.id)
│   └── budgets (user_id → users.id, category_id → categories.id)
```



Table Highlights & Soft Delete Strategy

Table	Key Design Choices
users	Stores hashed passwords only (bcryptjs, 10 salt rounds). Email is unique.
categories	Supports <i>income</i> or <i>expense</i> type. Soft-deleted with <i>is_deleted</i> flag.
transactions	Linked to both user and category. Soft-deleted so historical budget data is preserved.
budgets	A unique constraint on (<i>user_id</i> , <i>category_id</i> , <i>month</i> , <i>year</i>) prevents duplicate budgets per period.

Rather than ***DELETE*** records, we set *is_deleted* = ***TRUE***

This means:

- A deleted category does not break historical transaction records.
- Budget reports remain accurate even if a category is later removed.



Key Frontend Components (Contd.)

The frontend is built with **React + TypeScript** and structured around pages, services, and shared components.

Routing & Guards

```
App.tsx
├── /login      → Auth.tsx    (public)
├── ProtectedRoute → checks JWT in localStorage
│   └── DashboardLayout → Sidebar + <Outlet />
│       ├── /dashboard → Dashboard.tsx
│       ├── /transactions → Transactions.tsx
│       ├── /categories → Categories.tsx
│       └── /budgets → Budgets.tsx
```

Key Components

Dashboard.tsx → The main visual page

axiosInstance.ts → A central Axios instance

Service Layer → This decouples API logic from UI



Key Components Breakdown

Dashboard.tsx

The main visual page. Fetches one endpoint (`/api/dashboard`) and renders:

- Four summary cards: Total Income, Total Expenses, Current Balance, Budget Usage
- Doughnut chart: Expense distribution by category
- Bar chart: Monthly income vs expenses (last 6 months)
- Bar chart: Budget vs actual spending (current month)
- Recent transactions table

axiosInstance.ts

A central Axios instance that:

- Reads the base URL from `VITE_API_BASE_URL` in `.env`
- Automatically injects the **Authorization: Bearer <token>** header on every request
- Handles **401** responses globally by clearing `localStorage` and redirecting to login

Service Layer

Each resource has a dedicated service file (`authService`, `transactionService`, `categoryService`, `budgetService`, `dashboardService`). This decouples API logic from UI components, making both easier to test and maintain.



API Design and Backend Structure (Contd.)

The API follows **RESTful conventions** with all routes prefixed under *'/api'*.

Endpoint Summary:

Method	Route	Description
POST	<i>/api/auth/register</i>	Register a new user
POST	<i>/api/auth/login</i>	Login, receive JWT
GET	<i>/api/dashboard</i>	Aggregated dashboard metrics
GET / POST	<i>/api/transactions</i>	List (filterable) / create
PUT / DELETE	<i>/api/transactions/:id</i>	Update / soft-delete
GET / POST	<i>/api/categories</i>	List / create
PUT / DELETE	<i>/api/categories/:id</i>	Update / soft-delete
GET / POST	<i>/api/budgets</i>	List / create
PUT	<i>/api/budgets/:id</i>	Update amount

Backend Folder Structure & Security

Backend Folder Structure

```
src/
├── config/
│   └── db.js      ← MySQL connection pool (from .env)
├── middleware/
│   └── authMiddleware.js ← JWT verification, attaches req.user
├── controllers/
│   ├── authController.js
│   ├── categoryController.js
│   ├── transactionController.js
│   ├── budgetController.js
│   └── dashboardController.js
├── routes/
│   ├── authRoutes.js ← input validation + controller binding
│   ├── categoryRoutes.js
│   ├── transactionRoutes.js
│   ├── budgetRoutes.js
│   └── dashboardRoutes.js
└── app.js      ← Express setup, route mounting
```

Security

- Passwords are hashed with **bcryptjs** before storage.
- JWTs are signed with a secret from ***process.env.JWT_SECRET*** and expire in 1 hour.
- All private routes pass through the ***authMiddleware*** which verifies and decodes the token before the controller runs.
- Input validation is enforced via **express-validator** on every POST/PUT route before it reaches the controller.



Challenges Faced and How They Were Solved

1. Aggregating dashboard data efficiently

- **Problem:** The dashboard requires data from multiple tables (transactions, budgets, categories). Fetching them separately would result in several round trips and complex frontend composition.
- **Solution:** Created a single ***dashboardController*** endpoint (*GET /api/dashboard*) that runs all queries server-side and returns one structured JSON object with ***financialSummary***, ***visualInsights***, and ***recentTransactions***. This keeps the frontend simple and the data consistent.

2. Environment variable consistency

- **Problem:** Hardcoded fallback values (*http://localhost:5000/api*) in ***db.js*** and ***axiosInstance.ts*** meant misconfigured environments would silently connect to wrong databases or APIs.
- **Solution:** Removed all fallback values so the application fails fast with a clear error if ***.env*** is missing or incomplete. Configuration is strictly environment-driven.

3. Preventing duplicate budgets

- **Problem:** Without a constraint, a user could accidentally create two budgets for the same category in the same month.
- **Solution:** Added a database-level ***UNIQUE KEY*** on (***user_id***, ***category_id***, ***month***, ***year***) in the ***budgets*** table. The backend catches the ***ER_DUP_ENTRY*** MySQL error and returns a user-friendly 400 response.



Possible Future Improvements

- **Authentication** - Add refresh tokens and persistent sessions to extend JWT expiry without re-login
- **Recurring Transactions** - Allow users to schedule repeating income/expense entries (e.g., monthly salary)
- **Export** - Generate PDF or CSV reports of transactions and budget summaries
- **Notifications** - Alert users when spending approaches or exceeds a budget limit
- **Multi-currency** - Support multiple currencies with real-time exchange rates
- **Testing** - Add unit tests (Jest) for controllers and integration tests for API routes
- **Deployment** - Containerise with Docker and deploy backend on a cloud platform (e.g., Railway, Render)
- **Mobile App** - Extend the API to power a React Native mobile client using the same endpoints



Thank
you

